Analyzing Class Dependencies in Java Bytecode for Graph-Based Software Representation

Christian-Alexandru Dragoi
Faculty of Automation and Computers
Politehnica University Timisoara
Timisoara, Romania
christian.dragoi@student.upt.ro

***Abstract*—Understanding the structure of large object-oriented software systems is an important task in software maintenance and program comprehension. One approach to addressing this problem is representing software systems as dependency graphs, where classes are modeled as nodes and their relationships as edges. Such representations can support architectural analysis and the identification of key classes.**

This paper outlines a study of class dependency extraction from compiled Java bytecode. The study focuses on identifying which dependency types can be recovered from Java Archive (JAR) files and examining existing tools for bytecode analysis. A research question and an evaluation methodology are proposed to guide the development of a graph-based representation of Java software systems.

***Index Terms*—Java bytecode, static analysis, class dependencies, dependency graphs, program comprehension, software architecture**

I. Introduction

Large object-oriented software systems often contain hundreds or thousands of interconnected classes. Understanding the relationships between these classes can be difficult, especially when documentation is incomplete or does not reflect the current implementation.

Program comprehension and software architecture reconstruction attempt to address these difficulties by extracting structural information directly from software systems. One possible approach is to represent a system as a dependency graph, where nodes correspond to classes or interfaces and edges describe dependencies between them.

Such graphs can subsequently be analyzed using structural metrics and graph-based algorithms to identify important components of a system, commonly referred to as key classes [1].

Existing approaches frequently rely on source code analysis to extract these relationships. However, Java applications are commonly distributed as compiled JAR files, which contain Java bytecode rather than source code.

This creates an opportunity to investigate which structural dependencies can be recovered directly from compiled artifacts.

The objective of this paper is to establish a theoretical foundation for extracting class-level dependencies from Java bytecode and representing them as a graph.

II. State of the Art

A. Key Class Identification

The identification of key classes is an important research direction in software maintenance and program comprehension.

Sora and Chirila [1] proposed an approach for identifying key classes using static analysis and graph-based metrics. Their work represents software systems as graphs and investigates different measures of class importance, including direct connections and network centrality.

Their experimental evaluation indicates that weighted connection metrics and adapted PageRank measures can be effective for identifying structurally important classes.

A different approach was presented by Vale and Maia [2], who proposed Keecle, a technique based on dynamic analysis and method invocation traces.

While static analysis examines the structure of software without executing it, dynamic analysis considers relationships observed during execution.

Both approaches demonstrate the importance of class relationships for understanding software architecture.

B. Class Dependency Types

Sora and Chirila [1] distinguish several categories of static dependencies between object-oriented classes.

These include inheritance, interface implementation, field declarations, method parameters, return types, local variables, field accesses, method invocations, object instantiation, type casting and generic type bindings.

A class may depend on another class through multiple mechanisms simultaneously. For example, a class may declare a field of another type while also invoking methods belonging to that type.

Consequently, a dependency graph may need to preserve both the direction and the type of each relationship.

Furthermore, different dependency types may have different structural significance. The use of weighted edges provides a possible mechanism for representing these differences [1].

C. Java Bytecode Analysis

Java source code is compiled into class files containing information defined by the Java Virtual Machine (JVM) specification [3].

Class files contain structural metadata, including class names, superclass information, implemented interfaces, fields and method descriptors.

They also contain a constant pool and may contain bytecode instructions describing method invocations, field accesses and object creation.

However, not all source-level information is necessarily preserved after compilation.

For example, local variable metadata may be absent when the corresponding optional debug attributes are not included. Additionally, dynamically resolved relationships, including certain uses of reflection, may not be reliably identified through static analysis alone.

These characteristics must be considered when extracting dependency information from compiled Java applications.

D. Existing Bytecode Analysis Tools

Several tools and libraries provide mechanisms for examining Java bytecode.

ASM [4] is a bytecode manipulation and analysis framework that allows programmatic access to class structures and bytecode instructions.

Apache Commons BCEL [5] provides an alternative API for reading, analyzing and modifying Java class files.

The JDK Class-File API [6] is another relevant solution, providing a standard API for parsing and examining class files.

Additionally, the JDK includes tools such as `jdeps`, which identifies dependencies between classes and packages, and `javap`, which disassembles compiled class files [7], [8].

These solutions offer different levels of abstraction and functionality.

Their suitability for constructing a detailed class dependency graph requires further investigation.

III. Research Question

The study aims to investigate the feasibility of reconstructing class-level dependencies directly from compiled Java applications.

The main research question is:

> *RQ1: Which types of class-level dependencies can be reliably extracted from Java bytecode, and what limitations affect their representation in dependency graphs?*

To address this question, the study will examine the dependency categories identified in previous research and the information available in Java class files.

A secondary objective is to identify which existing bytecode analysis tools are suitable for supporting this extraction process.

IV. Proposed Methodology

The proposed study will combine documentation analysis with a small experimental evaluation.

A. Dependency Classification

The first stage will examine the dependency types presented in the literature [1].

Each dependency category will be mapped to the corresponding information available in the JVM class file format.

The dependencies will be classified according to whether they can be extracted directly, require additional analysis, or cannot be reliably recovered using static bytecode analysis alone.

B. Tool Analysis

The second stage will investigate existing Java bytecode analysis libraries, particularly ASM, BCEL and the JDK Class-File API.

The tools will be compared based on their ability to access class metadata, method signatures, bytecode instructions and dependency-related information.

Additional criteria will include API complexity and suitability for integration into a dependency extraction application.

C. Planned Experimental Evaluation

A small Java application containing representative dependency relationships will be developed and compiled into a JAR file.

The application will contain examples of inheritance, interface implementation, field dependencies and method calls.

The compiled classes will be examined using existing bytecode analysis tools.

The extracted information will then be compared with the dependencies explicitly defined in the source code.

This comparison will help determine which dependency categories can be recovered and which information may be lost or require additional analysis.

V. Results and Discussion

VI. Conclusions and Future Work

References

[1] I. Sora and C.-B. Chirila, "Finding key classes in object-oriented software systems by techniques based on static analysis," *Information and Software Technology*, vol. 116, Art. no. 106176, 2019. doi: 10.1016/j.infsof.2019.106176.

[2] L. do Nascimento Vale and M. de Almeida Maia, "Key classes in object-oriented systems: Detection and assessment," *International Journal of Software Engineering and Knowledge Engineering*, vol. 29, no. 10, pp. 1439–1463, 2019. doi: 10.1142/S0218194019500451.

[3] T. Lindholm, F. Yellin, G. Bracha, A. Buckley, and D. Smith, *The Java Virtual Machine Specification*, Java SE 26 Edition, Oracle, 2026. [Online]. Available: https://docs.oracle.com/javase/specs/jvms/se26/html/

[4] OW2 Consortium, "ASM: Java bytecode manipulation and analysis framework." [Online]. Available: https://asm.ow2.io/

[5] Apache Software Foundation, "Apache Commons BCEL." [Online]. Available: https://commons.apache.org/proper/commons-bcel/

[6] Oracle, "Package java.lang.classfile," *Java SE 26 API Specification*. [Online]. Available: https://docs.oracle.com/en/java/javase/26/docs/api/java.base/java/lang/classfile/package-summary.html

[7] Oracle, "The jdeps command," *JDK 26 Tool Specifications*. [Online]. Available: https://docs.oracle.com/en/java/javase/26/docs/specs/man/jdeps.html

[8] Oracle, "The javap command," *JDK 26 Tool Specifications*. [Online]. Available: https://docs.oracle.com/en/java/javase/26/docs/specs/man/javap.html